

NYPC 리플레이

리플레이 작성은 직접 하였으며 문제의 해결에 사용한 AI는 Gemini인것을 미리 밝힌다.

챌린지

그리디

우선은 LLM과의 대화를 통하여 같은 column내에서 $A_i \rightarrow B_i$ 로 향할 때의 $A_i > B_i$ 혹은 그 역에 대해서 제대로 분배하는것부터 하기로 시작하였다.

가장 간단하게 구간 $[0, i]$ 가 요구분을 초과한다면 $[i+1, N]$ 은 무조건적으로 필요를 하게 되고, 이는 Flow Right의 증가를 뜻한다.라는 성질을 이용해서 옆으로 보내야 하는 종류 두가지를 계산하였다.

일종의 그리디로 처음에 구현을 하였다.

이때 내 코드가 이상한 점이 많았다는 생각을 하였다. 그 이유는 계속 답이 같게 나왔기 때문이다. 그래서 D만 나오는 코드를 돌려보니 내 코드가 잘못됐다는 것을 깨닫게 되고 일단 재구현 해보았다.

그 후엔 그래도 처음보다는 점수가 더 괜찮게 나왔었다. 그 이후에는 조금 더 최적화를 했던거 같다. 층에 따라서 LEFT로 옮기는 연산과 RIGHT로 옮기는 연산을 분리하여 더 원하는대로 층을 분리 시켰다.

다음 아이디어는 R의 크기였다. R이 $C+10$ 이여도 점수는 $2^{**}10(1024)$ 밖에 오르지 않는다. 즉 일정의 R의 증가는 최선이 당연히 아니겠지만 오차를 줄이는 방향으로써는 깔끔하다고 생각하였다. 이때는 그냥 $C+2$ 로 저렴하게 만들었다.

그 이후에는 여러 그리디한 방법들을 최대한 많이 사용해보았다. 크게 유의미한 점수 변화는 없었다.

다음 유의미한 변화가 생겼던 부분이 있었는데, 이때는 중간까지는 최대한 씨앗을 퍼뜨리고, 그 후에는 분배를 하는식으로 알고리즘을 고안하였다. 그 이유는, 목표치가 한쪽으로 너무 몰아져있으면, 그 목표치까지 가는 과정이 더 어렵다고 생각하였기 때문이다. 그 후엔 바로 옆칸만 보는것이 아니라 옆의 전체 구간을 바라보는 아이디어를 적용해 보았던거 같다.

그 후에 오차를 더 줄일 수 있던 아이디어는 옆으로 갔을때의 이득을 계산하는 아이디어였던거 같다. 즉 현재 넘기지 않았을때와 넘겼을때의 오차가 누가 더 큰지 계산을 하여서 더 작은것의 행동을 개시하는거였다.

휴리스틱의 도입

이때까지는 그리디였다

그 다음부터는 조금더 휴리스틱적 접근을 하였다. 이 때 담금질 기법을 사용해야겠다고 생각을 했는데, 파이썬으로는 시간이 남아나질 않을거 같다고 생각해 C++로 갈아탔다. 초반에는 조금 오차가 커지더라도 후반에는 오차를 줄이는 방향으로 고안을 했었던거 같다. 기존의 그리디랑 섞어서 해보았지만 오히려 더 악화된거 같아 그냥 순수하게 담금질만 하기로 했다.

초반의 담금질은 거의 사실상 생 랜덤이었어서 여러가지 방향성? 목표치? 같은것을 조금 추가해서 설정을 해 주었다. 그러더니 오차가 더 내려갔다. T의 진행도나 현재 열에 대한 오차, 혹은 현재 행의 위치 등을 계산하였었다.

그 후부터 코드에 계속해서 개선이 되질 않아서 열심히 읽어보았다. 읽어보니

```
/ CMD_DL 인 경우
calc_next[c] += keep;
calc_next[c - 1] += give;
```

라는 부분이 있었는데, 이때 calc_next C와 clac_next C-1은 대응되는 관계가 아닌 대각선 관계인것을 발견해서 고쳐주었다. 그러니 오차가 더 많이 줄어들었다. 이때 채점 결과는 약 67만점이었다.

그 다음에는 내가 사실상 핵심으로 사용한 방법이 있는데 바로 리세마라를 돌리는 것이었다. 쉽게 말해서 리트를 여러번 돌려서 최적의 결과를 찾아내는것이다. 그것을 위해서 휴리스틱은 최대한 빨리 끝내고, 1.85/횟수 최적을 찾아내 뽑아내는 전략이었다. 이 전략이 가장 큰 점수변화를 주었다 생각하였다.

그 다음부터는 이것저것 그 코드에서 추가해볼수 있는 것들을 최대한 추가해보았던거 같다. 가장 많이 했던 것은 담금질을 하기 위한 초기 temp에 대한 것과, 최적의 R을 찾아내는거였던거 같다.

아무튼 여러가지를 더 시행해보고 코드는 담금질 하고, 리세마라, BFS를 통한 시뮬레이션을 통해서 현재 코드를 만들었다.

가장 최근에 제출한 코드는 R을 C+8,9,10 으로 하고 초기 온도를 3000으로 하는 코드와,

R을 C+7로 하고 초기 온도를 5000으로 하는 코드를 병합해서 사용하였다. 큰 이유는 없고 전자는 가장 나쁘지 않은 퍼포를 뽑아냈던거 같고, 후자는 전자에서 가장 약했던 케이스에서 그나마 적절한 점수를 뽑아내 두개를 병합한 코드를 사용하였다. 사실 근데 크게 다른건 없고 그저 초기값의 세팅만 차이가 있었다.

마지막으로는 여러 중간평가의 결과들을 통해서 값의 세팅을 좀더 정교하게 하였다.

마지막날에 중간평가를 보았는데 퍼포먼스가 좀 안좋게 나와서 다 조금 수정을 하여 세팅을 했다.

다람쥐나 햄스터가 과부하 걸리는거를 최대한 줄이고,
씨앗이 꼬이지 않게 DAG를 세팅해주고,

명령어 DLR 등을 DRL같은것들로 조금더 추가해주었다.

또한 재가열 같은걸 하거나, 무작위를 버리는 방식등 여러가지 방법을 시도해보았다.

스텝 업

자명한것은 넘길 것이며 특히 어려웠던거나 좀 재밌었던 발견을 적을 예정이다.



이 방법을 통해서 씨앗을 200 : 100 : 200으로 효율적이게 분배할 수 있다는것을 알게되었다. 재밌었던 풀이였다.



왜인지는 모르겠지만 이렇게 하면 생으로 분배하는것보다 1 더 빠르게 분배할 수 있다. 그래서 점수를 조금 더 많이 깎을 수 있게 된 기회였다.

14, 15번은 정확한 풀이를 찾아내지는 못하였지만 내 기존의 챌린지코드를 통하여 R=C로 고정시키고 나온 결과를 통하여 조금씩 수정을 하며 결과를 냈던거 같다. 물론 내 손으로도 직접 몇번 시뮬레이션을 돌려보기도 하였다.